

Advanced SQL Queries (Window Functions & Aggregation)

1. Fetch top 3 highest salaries department-wise.

Explanation: This requires a window function. We use **DENSE_RANK()** partitioned by department_id and ordered by salary descending. We then select rows where this rank is 3 or less.

Query:

```
SELECT department_id, name, salary
FROM (
    SELECT
        *,
        DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary
        DESC) as salary_rank
    FROM Employees
) AS RankedSalaries
WHERE salary_rank <= 3;
```

2. Top N employees based on salary.

Explanation: We sort all employees by salary in descending order and use the **LIMIT** clause to restrict the result set to the top N rows. **Query (To fetch the top 10 employees):**

```
SELECT *
FROM Employees
ORDER BY salary DESC
LIMIT 10;
```

3. Find employees working under a specific manager.

Explanation: Assuming the Employees table has a manager_id column that refers to another employee's employee_id, we simply filter based on the specific manager's ID. **Query:**

```
SELECT *
FROM Employees
WHERE manager_id = 105; -- Replace 105 with the specific manager's ID
```

4. Retrieve employees who worked on Project A or B using UNION (no duplicates).

Explanation: We use **UNION** to combine two separate SELECT statements. UNION automatically performs a deduplication step, ensuring each employee appears only once.

Query:

```
SELECT employee_id, name FROM EmployeeProjects WHERE project_name =  
'Project A'  
UNION  
SELECT employee_id, name FROM EmployeeProjects WHERE project_name =  
'Project B';
```

5. Write a query to get the second highest salary (without LIMIT).

Explanation: We select distinct salaries and use a subquery combined with **MAX()**. We find the maximum salary that is still *less than* the absolute maximum salary, effectively finding the second highest.

Query:

```
SELECT MAX(salary)  
FROM Employees  
WHERE salary < (SELECT MAX(salary) FROM Employees);
```

6. Running total of sales amount ordered by date.

Explanation: We use the **SUM()** window function, ordering by date and defining the window frame to include all preceding rows up to the current row. This calculates the cumulative sum.

Query:

```
SELECT  
    sale_date,  
    amount,  
    SUM(amount) OVER (ORDER BY sale_date ROWS UNBOUNDED PRECEDING) AS  
running_total  
FROM Sales  
ORDER BY sale_date;
```

7. Assign a row number to each employee within their department by salary desc.

Explanation: We use the **ROW_NUMBER()** window function, partitioning the data by department_id and ordering the results by salary descending. **Query:**

```
SELECT  
    department_id,  
    name,  
    salary,  
    ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC)  
AS dept_rank  
FROM Employees;
```

8. Return the top 2 salaries per department using window functions.

Explanation: We use **DENSE_RANK()** partitioned by department to assign ranks. We then filter the results in an outer query where the calculated rank is 2 or less. **Query:**

```
SELECT department_id, name, salary
FROM (
    SELECT
        *,
        DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary
        DESC) as salary_rank
    FROM Employees
) AS RankedSalaries
WHERE salary_rank <= 2;
```

9. Count orders (total + amount) per status in one row per status.

Explanation: We use **GROUP BY** to categorize results by order_status and use aggregate functions **COUNT()** and **SUM()** on the grouped data. **Query:**

```
SELECT
    order_status,
    COUNT(order_id) AS total_orders,
    SUM(order_amount) AS total_sales_amount
FROM Orders
GROUP BY order_status;
```

10. Customers who placed orders in every month of the current year.

Explanation: We find the count of distinct months in the current year where orders were placed by a customer. We then compare this count to the total number of months that have passed in the current year.

Query:

```
SELECT customer_id
FROM Orders
WHERE YEAR(order_date) = YEAR(CURDATE())
GROUP BY customer_id
HAVING COUNT(DISTINCT MONTH(order_date)) = MONTH(CURDATE());
```

-- This assumes the current month is complete and does not filter for future months.

11. Pivot orders → show total orders for statuses (PENDING, SHIPPED, CANCELLED).

Explanation: We use the **SUM()** aggregate function combined with **CASE** expressions to pivot the data. The CASE statement converts the order status into a count of 1 only if the condition is met, otherwise it's 0. **Query:**

```
SELECT
    SUM(CASE WHEN order_status = 'PENDING' THEN 1 ELSE 0 END) AS
    Total_Pending,
    SUM(CASE WHEN order_status = 'SHIPPED' THEN 1 ELSE 0 END) AS
    Total_Shipped,
    SUM(CASE WHEN order_status = 'CANCELLED' THEN 1 ELSE 0 END) AS
    Total_Cancelled
FROM Orders;
```

12. Categorize employees using CASE: LOW (<20k), MID (20k–50k), HIGH (>50k).

Explanation: We use the **CASE** expression to create a new derived column (salary_band) and apply conditional logic based on the value of the salary column. **Query:**

```
SELECT name, salary,
    CASE
        WHEN salary < 20000 THEN 'LOW'
        WHEN salary BETWEEN 20000 AND 50000 THEN 'MID'
        ELSE 'HIGH'
    END AS salary_band
FROM Employees;
```

13. Count how many employees fall into each salary band.

Explanation: We combine the conditional logic from the previous CASE expression with the **GROUP BY** clause to count the employees in each resulting band. **Query:**

```
SELECT salary_band, COUNT(*) AS employee_count
FROM (
    SELECT
        CASE
            WHEN salary < 20000 THEN 'LOW'
            WHEN salary BETWEEN 20000 AND 50000 THEN 'MID'
            ELSE 'HIGH'
        END AS salary_band
    FROM Employees
) AS BandedEmployees
GROUP BY salary_band;
```

14. Find top 3 most sold products by total quantity.

Explanation: We group the sales records by product ID, calculate the total quantity sold using **SUM()**, order the results in descending order, and restrict the output using **LIMIT**. **Query:**

```
SELECT product_id, SUM(quantity) AS total_sold
FROM Sales
GROUP BY product_id
ORDER BY total_sold DESC
LIMIT 3;
```

15. Records in table A not present in table B (LEFT JOIN + NOT IN versions).

Explanation (LEFT JOIN): We perform a **LEFT JOIN** from A to B. This returns all rows from A, including those with no match in B (where b.id will be NULL). We filter by b.id IS NULL. **Query (LEFT JOIN):**

```
SELECT a.*
FROM TableA a
LEFT JOIN TableB b ON a.id = b.id
WHERE b.id IS NULL;
```

Explanation (NOT IN): We select all rows from A where the id value is **NOT IN** the set of id values found in Table B.

Query (NOT IN):

```
SELECT *
FROM TableA
WHERE id NOT IN (SELECT id FROM TableB);
```

16. Find orphan child records where foreign key does not match any parent.

Explanation: Assuming ChildTable.parent_id is the foreign key, we perform a **LEFT JOIN** from the ChildTable to the ParentTable. We then filter for records where the joined ParentTable.id is **NULL**.

Query:

```
SELECT c.*
FROM ChildTable c
LEFT JOIN ParentTable p ON c.parent_id = p.id
WHERE p.id IS NULL;
```